



HUNTER2

OFFENSIVE SECURITY ENGINE

AUTHORIZED TESTING ONLY

FULL CAPABILITY REFERENCE

HOW IT HUNTS.

A plain-English map of **what the tool does**, every vulnerability class it tests, **how** it tests each one, how it behaves like a real human hacker, and the agents & tools that do the work — running identically in **Claude Code** and **OpenCode**.

11

phase end-to-end
hunt workflow

54

leveled playbooks
(flow · payloads · rules)

15

specialist agents
orchestrated

~55

vulnerability
classes covered

~15 automated scanner classes

~40 expert manual classes

2-account cross-tenant testing

Auto-chaining A → B → C

Compounding hunt memory



A deterministic scope checker gates every request. Destructive HTTP methods and report submission require explicit operator approval. The tool finds and guides — a human confirms.

THE END-TO-END HUNT

One command runs the full professional workflow

Run `hunt <target>` and the tool executes an eleven-phase pipeline — the same in Claude Code and OpenCode, which share one `tools/` engine.

01

Scope

In-scope check; every request gated

02

Recon

Subdomains, live hosts, crawl, tech, JS

03

Freshness

Diff vs last recon → hunt new first

04

Playbook ctx

Auto-index the right playbook per class

05

Scanners

Real payloads across automatable classes

06

Manual test

Agent works the disclosed-report checklist

07

Two-account

Cross-tenant IDOR / BOLA / privesc

08

Auto-chain

Each hit → sibling + A→B next-tests

09

Validate

Rejection gate + 7-question gate

10

Report

Impact-first platform-ready write-up

11

Learn

Record outcomes → next hunt is smarter

PHASE	WHAT HAPPENS	KEY TOOL(S)
• Scope	Confirm the asset is in-scope; every request is gated.	<code>scope_checker.py</code> · <code>/scope</code>
Recon	Subdomain enum, live hosts, URL crawl, tech fingerprint, JS extraction.	<code>recon_engine.sh</code> · <code>subfinder</code> / <code>dnsx</code> / <code>katana</code> / <code>gau</code> / <code>nuclei</code>
• Freshness	Diff vs last recon → new assets get hunted first (Rule 12).	<code>recon_diff.py</code>
Playbook context	Auto-index the right playbook per class → findings/<t>PLAYBOOKS.md.	<code>playbook_router.py</code>
Scanners	Fire real payloads across the ~15 automatable classes.	<code>*_scanner.py</code> <code>set</code> · <code>vuln_scanner.sh</code>
Playbook testing	Agent tests the ~40 knowledge classes by the disclosed-report checklist.	<code>real-world-playbooks</code> <code>skill</code>
Two-account	Cross-tenant IDOR / BOLA / payment / privesc with your two accounts.	<code>two_account_idor.py</code>
• Auto-chain	Each confirmed hit → sibling + A→B next-tests.	<code>chain_engine.py</code>
• Validate	Auto-kill invalid findings, then the 7-question gate.	<code>rejection_gate.py</code> · <code>validate.py</code>
• Report	Impact-first H1 / Bugcrowd / Intigriti / Immunefi report.	<code>report-writer</code> <code>agent</code> · <code>/report</code>
• Learn	Record worked / rejected / dead-ends → next hunt is smarter.	<code>hunt_memory.py</code>

— HOW IT TESTS LIKE A REAL HUMAN

Scanners find duplicates. Humans find the money bugs.

The tool encodes real-hunter discipline from [rules/hunting.md](#) and [rules/lead-commander.md](#) — the moves a scanner structurally cannot make.



Two-account testing

#1 MOVE

Log in as A and B, replay A's requests with B's auth to prove cross-tenant access. **Scanners cannot do this** — it's the single highest-ROI move.



The Sibling Rule

~30% OF PAID IDOR

For every working endpoint, test its siblings — **/export, /delete, /share, /v1/** ... This alone explains roughly a third of paid IDOR.



A → B signal method

SIGNAL

One confirmed bug means the dev made a **class** of mistake. Look for the same flaw nearby before writing the report.



Follow the money

HIGHEST ROI

Billing, credits, refunds, wallet and checkout get the most dev shortcuts — so they carry the **highest payout density**.



Impact-first

PRIORITIZE

Always ask "what's the worst thing if auth broke here?" — then **skip low-value surface** and spend time where it pays.



Hunt fresh / new

FRESHNESS

Features under **30 days old** have the weakest security. The freshness diff surfaces them so they get hit first.



Kill weak findings fast

VALIDITY RATIO

The rejection gate + 7-question gate stop invalid reports **before they cost you** validity ratio on the platform.



Depth over breadth

FOCUS

One target understood deeply beats ten scanned shallowly. Understanding, not coverage, is what converts.



Learn across hunts — it compounds

The tool remembers what worked and what got rejected, so every engagement makes the next one sharper. Knowledge doesn't reset between targets.

— EVERY VULNERABILITY CLASS — AND HOW EACH IS TESTED

Access control · Authentication · Injection

LEGEND **SCANNER** automated payload firing **PLAYBOOK** agent tests by hand vs disclosed-report checklist **TWO-ACCOUNT**

OOB **BROWSER / CHAIN**

Access control & authorization **the money classes**

4 classes

IDOR / BOLA **SCANNER** **TWO-ACCOUNT**

ID swap, mutation IDOR, API-spec-driven, and cross-account replay (A's request, B's token). Sibling rule on every hit. [h1_idor_scanner](#) · [apispec_idor](#)

BFLA — function-level auth **PLAYBOOK**

Low-priv user hitting admin / privileged endpoints; every sibling in the same controller. [api_security_scanner](#)

Mass assignment / BOPLA **PLAYBOOK**

Inject extra JSON fields — [role](#), [is_admin](#) — the server trusts blindly.

Privilege escalation **PLAYBOOK** **TWO-ACCOUNT**

Role transitions, horizontal + vertical, hidden admin routes.



Authentication

7 classes

Auth / session **PLAYBOOK**

Logout invalidation, fixation, token entropy, concurrent sessions, cookie flags. [auth-session](#)

OAuth / OIDC **SCANNER**

redirect_uri manipulation, state reuse / CSRF, PKCE, code reuse, token leak. [h1_oauth_tester](#)

SAML / SSO **PLAYBOOK**

XML signature wrapping (XSW), NameID comment injection, signature stripping. [/auth-hunt](#)

JWT **SCANNER**

alg:none forgery, RS256→HS256 confusion, weak-secret crack, kid injection. [jwt_scanner](#)

MFA / 2FA **PLAYBOOK**

OTP brute (with lockout check), backup-code abuse, flow bypass.

Reset-password / registration **PLAYBOOK**

Token poisoning, host-header injection into reset links.

Account takeover **PLAYBOOK** **CHAIN**

The ATO taxonomy — 9 paths, assembled from chained primitives.



Injection

7 classes

SQLi **SCANNER**

Error / boolean / time-based via nuclei, ghauri, sqlmap. [vuln_scanner.sh](#)

NoSQLi **SCANNER**

[\\$ne](#) / [\\$gt](#) / [\\$regex](#) operator auth-bypass, [\\$where](#) blind. [nosqli_scanner](#)

Command injection / RCE **PLAYBOOK** **OOB**

OOB callback confirm — never destructive.

SSTI **SCANNER**

[{{7*7}}](#) / [\\${7*7}](#) math canaries per engine (Jinja / Twig / FreeMarker / ERB).

XXE **SCANNER** **OOB**

File-read + OOB blind exfil. [xxe_scanner](#) · [oob_listener](#)

LFI / path traversal → RCE **PLAYBOOK**

php://filter, log / session poisoning, wrappers.

Deserialization **PLAYBOOK** **OOB**

Java / PHP / Python / .NET gadgets, OOB confirm.

— EVERY VULNERABILITY CLASS — CONTINUED

Client-side · Server-side & infra · Logic · AI · Web3



Client-side & web

7 classes

XSS — reflected / stored / DOM

SCANNER

BROWSER

dalfox + a real headless Chromium harness — only [\[CONFIRMED\]](#) when the browser executes the payload. [dom_xss_harness](#)

CORS

SCANNER

Origin reflection, null origin, suffix / prefix bypass, credentialed read. [cors_scanner](#)

HPP · postMessage · clickjacking · open-redirect

SCANNER

PLAYBOOK

[hpp_postmessage_scanner](#) + playbooks.

CRLF / host-header injection

SCANNER

Encoded CRLF, Set-Cookie canary. [crlf_scanner](#)

CSRF

SCANNER

SameSite + token-entropy + validation check. [csrf_scanner](#)

Prototype pollution

SCANNER

BROWSER

Server [__proto__](#) probes + client gadget. [prototype_pollution_scanner](#)

WebSocket — CSWSH

SCANNER

Cross-site hijack, origin + message-auth. [websocket_scanner](#)



Server-side & infra

9 classes

SSRF

SCANNER

OOB

Sink discovery (webhook / import / preview), filter bypass, cloud metadata [169.254.169.254](#), blind OOB. [/ssrf-chain](#)

Web cache poisoning

PLAYBOOK

Unkeyed headers reflected into cached responses.

Secrets / CI-CD

SCANNER

PLAYBOOK

JS / source secret grep, [pull_request_target](#) + expression-injection. [secrets_hunter](#) · [sast_scan](#)

Kubernetes

PLAYBOOK

Unauth API server, kubelet, anonymous RBAC, exposed dashboard. [/k8s-audit](#)

CVE sweep

SCANNER

nuclei templates. [cve_scan](#) · [/scan-cves](#)

Request smuggling

PLAYBOOK

CL.TE / TE.CL / H2.CL desync probes.

Subdomain takeover

SCANNER

Dangling CNAME + claimable-service check + PoC page. [takeover_scanner](#)

Cloud storage

SCANNER

S3 / GCS / Azure bucket enum + takeover. [cloud_bucket_enum](#)

Port / service scan

SCANNER

naabu for non-web services — Redis, Docker API, DBs. [port_scanner](#)



Business logic & timing

2

Business logic

PLAYBOOK

REASONING

Negative quantity, price tampering, coupon stacking, step-skipping, quota bypass. [business-logic-hunter](#)

Race conditions

SCANNER

Parallel / pipelined fire on redeem / refund / transfer / vote. [hl_race](#) · [/race](#)



AI / LLM & Web3

2

LLM / agentic

SCANNER

Prompt injection, jailbreak, system-prompt leak, data exfil, indirect injection, RAG / MCP poisoning, ASI01-ASI10. [llm_redteam](#)

Smart contracts

PLAYBOOK

10 EVM classes (reentrancy, oracle, access control, accounting desync) + Solana SPL + meme-coin rug checks. [web3-auditor](#)

— THE AGENTS — WHO DOES THE WORK

One orchestrator, fourteen specialists

The primary **hunter** orchestrates; each specialist owns its domain and reports leads back up the chain.

**hunter**

Primary orchestrator — drives the full pipeline and dispatches every specialist below.

**recon-agent**

Subdomain enum + live-host discovery (Chaos / subfinder / dnsx / httpx).

**recon-ranker**

Ranks attack surface + hunt memory into a prioritized plan.

**api-hunter**

REST / GraphQL / gRPC vs OWASP API Top 10 (2023) + two-account IDOR.

**business-logic-hunter**

The reasoning bugs scanners can't find — logic & abuse-of-function.

**race-hunter**

TOCTOU / race conditions — coupons, wallet, limits.

**cloud-hunter**

AWS / GCP / Azure exposed assets, buckets, metadata SSRF.

**llm-hunter**

Chatbots / copilots / RAG / agents — prompt injection + ASI framework.

**credential-hunter**

Password-spray pipeline — wordlist-gen + breach-check + OSINT.

**novel-vuln-reasoner**

Unknown / novel classes with no signature — deep code & behavior review.

**chain-builder**

Given bug A, finds B and C to chain for higher severity / payout.

**token-auditor**

Meme-coin / token security — mint, honeypot, fee manipulation.

**web3-auditor**

Smart-contract audit — 10 classes by frequency.

**validator**

7-Question Gate + 4-gate checklist — kills weak findings.

**report-writer**

Impact-first H1 / Bugcrowd / Intigriti / Immunefi reports.

**autopilot**

Autonomous full loop — scope → recon → rank → hunt → validate → report, with safety rails.

— THE PHASE-1/2 ENGINE

What makes it "active", not just knowledgeable

Built in the EXTREME upgrade, these components make the tool **act** on its knowledge — turning playbooks into moves.

playbook_router.py

Resolves any class / alias → the right playbook; auto-writes **PLAYBOOKS.md** per hunt.

two_account_idor.py

Cross-tenant replay (A's request, B's auth); safe methods by default; never logs tokens.

rejection_gate.py

Auto-kills always-rejected findings — public keys, self-XSS, theoretical, out-of-scope.

chain_engine.py

Sibling rule + A→B table → ranked next-tests after each confirmed hit.

hunt_memory.py

Records worked / rejected / dead-ends → next hunt loads them, so the tool compounds.

recon_diff.py

Flags new assets since last recon → hunt fresh first.



Safety infrastructure — always on

- **scope_checker.py** — deterministic scope gate on every single request.
- **credential_store.py** — .env, masked, never in transcripts.
- **AutopilotGuard** — circuit breaker + rate limiter + SafeMethodPolicy.
- **Audit log + pattern DB** — with 10 MB rotation.

— WHAT IT WILL NOT DO

Honest limits

A tool you can trust states its boundaries plainly. Here are Hunter2's.



It does **not** auto-find bugs with zero human input — it finds & guides; **you confirm**.



The ~40 playbook classes have **no dedicated auto-scanner** — the agent tests them by hand with expert guidance (this finds the high-value bugs anyway).



It won't test out-of-scope assets, run DoS, social-engineer, exfiltrate real user data, or enter your credentials / solve CAPTCHAs — **it stops and asks**.



High-value access-control bugs need **your two own test accounts** in `.env`.

— IN ONE LINE

Point it at an authorized target and it runs the **full hunt** — recon, the right playbook per lead, real scanners for the automatable classes, expert manual methodology for the rest, two-account testing for the money bugs, auto-chaining, hard validation, a clean report — and it **gets smarter every hunt**.

Claude Code
SHARED TOOLS/ ENGINE



OpenCode
IDENTICAL BEHAVIOR

■ SAME BEHAVIOR IN BOTH RUNTIMES · AUTHORIZED TESTING ONLY